TD 4 : Définition de données, Contraintes

Définition de données, Contraintes

2026-10-23

L3 MIASHS
Université Paris Cité
Année 2025
Course Homepage
Moodle

Prérequis

- schéma `world`,
- cours Création de tables, Insertion, Suppression, Mises à jour.
- cours Contraintes, Check, Primary Key, Foreign Key.

Contraintes SQL

Documentation PostgreSQL.

Contraintes CHECK.

Cette contrainte permet de vérifier que les colonnes d'une ligne donnée vérifient une certaine condition. Par exemple, on pourrait vouloir vérifier que la valeur de la colonne `percentage` de la table `countrylanguage` est inférieure à 100 ou que la valeur de la colonne `return_date` de la table `rental` est soit `NULL` ou supérieure à `rental_date`. Ces contraintes se définissent lors de la création de la table. Par exemple :

```
CREATE TABLE solde(
    produit TEXT,
    reduc_pourcentage INT,
    CHECK (reduc_pourcentage<=100 AND reduc_pourcentage%10=0)
);
```

On peut ajouter une contrainte CHECK à une table qui existe déjà. Par exemple :

```
ALTER TABLE solde ADD
    CHECK (reduc_pourcentage >=0);
```

Il est préférable de vérifier qu'une table existe avant de tenter d'ajouter une contrainte :

```
ALTER TABLE IF EXISTS solde ADD
    CHECK (reduc_pourcentage >=0);
```

ou bien en nommant la contrainte :

```
ALTER TABLE IF EXISTS solde
    ADD CONSTRAINT reduc_positive
       CHECK (reduc_pourcentage >=0);
```

Contraintes de colonne `DEFAULT / NOT NULL / UNIQUE`

Ces contraintes s'appliquent à une colonne en particulier. La contrainte `DEFAULT` spécifie une valeur par défaut pour remplir une colonne lorsque sa valeur n'est pas spécifiée, `NOT NULL` que la valeur de la colonne ne peut pas être `NULL` et `UNIQUE` que deux lignes différentes de la table ne peuvent pas avoir la même valeur sur les colonnes indiquées. Par exemple :

```
CREATE TABLE membre(
    nom VARCHAR(50) NOT NULL,
    prenom VARCHAR(50) NOT NULL,
    date_inscription DATE DEFAULT NOW(),
    UNIQUE(nom, prenom)
);
```

Contraintes de table `PRIMARY/FOREIGN KEY`

- Formellement, la contrainte `PRIMARY KEY` est équivalente à `UNIQUE` et `NOT NULL`. C'est une ou plusieurs colonnes qui permettent d'identifier de façon unique chaque ligne. Dans une base de données bien conçue, chaque table doit avoir une clé primaire.
- On ne peut définir qu'une seule contrainte `PRIMARY KEY` sur une table.
- La contrainte `FOREIGN KEY` spécifie que la valeur d'une (ou de plusieurs) colonne contient des valeurs présentes dans une autre table. Par exemple, la colonne `capital` de `country` contient une valeur qui doit apparaître dans la colonne `id` de `city`.
- On peut définir plusieurs contraintes `FOREIGN KEY` sur une table.

Exemple :

```
CREATE TABLE IF NOT EXISTS membre(
    id INT,
    nom VARCHAR(50) NOT NULL,
    prenom VARCHAR(50) NOT NULL,
   PRIMARY KEY (id)
);

-- si la clé primaire ne contient qu'une colonne,
-- on peut aussi la définir au niveau de la colonne
-- idem pour la clé étrangère
CREATE TABLE IF NOT EXISTS amis(
    id1 INT PRIMARY KEY,
    id2 INT,
   FOREIGN KEY (id1) REFERENCES membre (id)
);
```

NB : ici, on a ajouté la clause `IF NOT EXISTS` pour éviter une erreur d'exécution si la table existe déjà. Dans ce cas, la requête ne fait rien. Si on veut recréer une table, il faut d'abord la supprimer.

On peut aussi ajouter ces contraintes après la définition de la table (il est alors préférable de vérifier que la table existe avant de tenter d'ajouter une contrainte) :

```
ALTER TABLE IF EXISTS amis ADD
    PRIMARY KEY (id1);
ALTER TABLE IF EXISTS amis ADD
    FOREIGN KEY (id2) REFERENCES membre;
```

Contraintes utilisant une fonction

Les contraintes `NOT NULL, DEFAULT, KEY, CHECK` portent uniquement sur une ligne tandis que la contrainte `UNIQUE` permet uniquement de tester des égalités entre lignes. On veut parfois vérifier une condition plus riche entre des lignes. Par exemple, si notre table contient des réservations d'hôtel entre `start_date` et `end_date`, on ne veut pas que deux réservations se chevauchent.

On peut alors utiliser une fonction dans une contrainte `CHECK` :

```sql
create table if not exists reservation(
   id int primary key,
   start_date date not null,
   end_date date not null check (end_date > start_date)
);

create or replace function compat (id_res int, start_res date)
returns boolean
language sql as
$$
   select count (*) = 0
   from reservation r
   where id_res <> r.id
      and start_res >=  r.start_date
      and start_res <  r.end_date;
$$
-- il est possible aussi d'utiliser daterange(start_date, end_date)

alter table if exists reservation
   add constraint check_compat check (compat (id,start_date)) ;

insert into reservation values(1,'01/01/2026','01/10/2026'); -- est accepté
insert into reservation values(2,'01/10/2026','01/12/2026'); -- est accepté
insert into reservation values(3,'01/05/2026','01/10/2026'); -- est refusé
```

> **!** **Mode de vérification des contraintes `CHECK`**
>
> Les contraintes `CHECK` sont vérifiées uniquement lors de l'insertion et de la mise à jour. Les données de la table utilisées par la fonction sont celles existantes avant l'insertion et la mise à jour. **Les paramètres de la fonction sont les attributs de la ligne que l'on veut insérer ou mettre à jour**.

Contraintes `EXCLUDE`

La clause `EXCLUDE` permet de définir des contraintes d'exclusion entre deux lignes quelconques d'une table, c.a.d. que pour tout couple de lignes de la table, la propriété est exclue.

On obtient une contrainte équivalente à la précédente avec la définition suivante :

```sql
create table if not exists reservation(
   id int primary key,
   start_date date not null,
   end_date date not null check (end_date > start_date),
   exclude using gist (
        daterange(start_date, end_date) with &&)
);
```

Explication :

- `GiST` définit un type d'index pour effectuer les recherches. `GiST` comme "Generalized Search Tree" désigne une catégorie d'arbres (cf. Wikipedia).
- `daterange` calcule un intervalle de dates.
- `&&` est l'opérateur de comparaison utilisé. En PostgreSQL, il s'agit du test de recouvrement entre deux `daterange`.
- `exclude using gist (daterange(start_date, end_date) with &&)` signifie que pour deux lignes `r1` et `r2`, il est interdit que les intervalles `daterange(r1.start_date, r1.end_date)` et `daterange(r2.start_date, r2.end_date)` se recouvrent.

Voir la documentation PostgreSQL.

Exercices

On va travailler sur une copie du schéma `world`. Il va donc falloir copier dans vos schémas les tables `world.country`, `world.city` et `world.countrylanguage`.

```
CREATE TABLE IF NOT EXISTS td_country (like world.country) ;
```

La clause `like` permet de copier la structure d'une table existante dans une nouvelle table avec les types des colonnes et les contraintes `NOT NULL`. Mais aucune autre contrainte.

Voir la documentation de la clause `like`

Par contre, l'option `including all` permet de copier toutes les contraintes SAUF celles qui font référence à d'autres tables (clé étrangère, ...) :

```
CREATE TABLE IF NOT EXISTS td_country (
    like world.country
    including all
) ;
```

> **Vérifier la définition d'une table existante avec dBeaver**
>
> Avec dBeaver, vous pouvez visualiser la définition d'une table avec toutes ses contraintes en développant le schéma qui la contient puis "tables" puis la table qui vous intéresse.
> Ensuite :
> - ou bien dans la table, développer "DDL" et vérifier la définition de la table en SQL.
> - Ou bien dans la table, développer "Colonnes", "Contraintes" et "Clés étrangères" pour visualiser toutes les contraintes définies sur la table.

Questions

> Modifiez le `search_path` pour inclure votre schéma personnel.

1. Créer dans votre schéma trois tables de mêmes structures que celles de `world`.
2. Insérer dans les nouvelles tables les valeurs des tables originelles du schéma `world`.
3. Recréer les contraintes qui n'ont pas été copiées pour obtenir des tables qui soient réellement identiques. Vérifier cette identité en utilisant la note précisée en début d'exercice.
4. Ajouter la contrainte : la colonne `percentage` de `td_countrylanguage` contient un réel compris entre 0 et 100.
5. Comment ajouter une contrainte `unique` sur le couple `(countrycode, language)` de la table `td_countrylanguage` ? Est-il utile d'ajouter cette contrainte à celles préalablement définies ?
6. Mettre à jour dans `td_city` les populations des capitales du continent `Africa` en les multipliant par 1,1. Après cette mise à jour, la population de Djibouti est 421 300.
7. Supprimer les pays de la table `td_country` où aucune langue n'est parlée et qui n'ont pas de capitale (utiliser des CTEs). (6 lignes supprimées).
8. Réécrire la requête d'insertion des données de la table `world.country` dans la table `td_country` en filtrant les lignes qui ne respectent pas les contraintes d'unicité par rapport aux données de la table `td_country` avant l'insertion. Après la suppression de la question précédente, réinsérer avec cette nouvelle requête, les données de la table `world.country`. Si votre requête est correcte, son exécution ne doit pas provoquer de message d'erreur et `td_country` doit contenir à l'issue les mêmes données que `world.country`.

NB : la valeur `NULL`est autorisée pour la colonne `capital`.

9. Ajouter la contrainte : dans `td_country`, pour chaque pays, il n'existe pas un autre pays de la même `region` qui n'est pas dans le même `continent`. Il s'agit d'une contrainte correspondant à la *dépendance fonctionnelle* `region → continent`, c.a.d. que tous les pays d'une même région doivent être sur un même continent.

Après la définition de la contrainte, tester l'insertion suivante :

```
insert into td_country(countrycode, name_country, continent, region, code2, localname, governmentfo
(
   select 'FR2', name_country, 'Asia', region, code2, localname, governmentform, population_country
   from td_country
   where countrycode = 'FRA'
);
```

Vous devez obtenir une message d'erreur indiquant que la contrainte définie n'est pas respectée.